Trabajo Final Integrador: Rubípticos!

Crípticos en Ruby

TTPS Opción Ruby

En este Trabajo Final Integrador (TFI), buscaremos que apliquen los conocimientos adquiridos en la materia para resolver un problema real. El objetivo de la implementación que deberán hacer es concebir un sistema que permita resolver las necesidades expuestas en este documento, como si se los hubiera pedido un potencial cliente o persona interesada en los servicios de desarrollo de software que ustedes proveen.

Contexto

Están a cargo de desarrollar un sitio de juego de crípticos, que son adivinanzas o acertijos que los jugadores pueden resolver para divertirse.

Requisitos técnicos

El desarrollo debe realizarse utilizando el siguiente stack tecnológico:

- La versión estable más reciente del *framework* Ruby on Rails (8.1.4 al momento de escribir este documento).
- Una versión reciente de Ruby (3.3 o superior), acorde a lo requerido por el framework.
- Una base de datos para dar soporte de persistencia. SQLite es ideal y suficiente. Adicionalmente, se les pide que no reinventen la rueda y cuando sea posible utilicen gemas bien establecidas que proveen funcionalidad habitual en una aplicación *web*.

Alcance

Cada grupo construye en Ruby on Rails un sitio web para jugar a **crípticos**: acertijos de palabras en español en los que una frase esconde una palabra detrás de un juego de palabras. El jugador entra desde un navegador web solo con un nombre, resuelve crípticos, suma puntos, mantiene una racha de días jugados y compite en una tabla de posiciones.

El sistema tiene tres partes:

- **Sitio público (frontend):** el juego, las rachas, la tabla de posiciones y un asistente (chatbot) que explica las reglas paso a paso.
- **Backend de administración:** los administradores ajustan la configuración del juego y mantienen el catálogo de cripticos.
- **Integración:** el catálogo de crípticos se sincroniza desde una API JSON publicada por la cátedra.

El TP cubre modelado con Active Record, validaciones, lógica de negocio, manejo de fechas, cookies, hotwire, autenticación de usuarios administradores, consumo de servicios HTTP y testing (deseable/opcional).

¿Qué es un críptico y cómo se juega?

Un críptico es una frase que lleva dos veces a la misma palabra: una vez por su significado y otra por un juego de palabras. Nacieron en los crucigramas británicos (*cryptic crosswords*). Cada críptico se juega suelto. El jugador lee un enunciado y escribe la palabra que esconde.

La gracia es que la frase se lee como algo normal, con sentido, pero hay que leerla como si fueran instrucciones. Por ejemplo, «Pato rosado esconde un animal de las corridas (4)» parece hablar de un pato, pero en realidad dice: «en las palabras *Pato rosado* está escondido un animal de las corridas, de 4 letras». Y ahí está: pa**TO RO**sado, TORO.

Las partes de un críptico

Parte	Qué es	En «Pato rosado esconde un animal de las corridas (4)»
Definición	Un sinónimo o descripción de la respuesta. Siempre está al principio o al final del enunciado.	un animal de las corridas
Indicador	La palabra que dice qué operación hacer.	esconde
Material	Las letras o palabras sobre las que se opera.	Pato rosado

Enumeración	La cantidad de letras de la respuesta, entre paréntesis. Las tildes no cuentan.	(4)

La definición sola ya alcanzaría para adivinar, pero suele ser ambigua: «un animal de las corridas» puede ser TORO o CABALLO. El juego de palabras es lo que confirma cuál es. Cuando las dos lecturas llevan a la misma palabra, la respuesta es segura.

Cómo se resuelve, paso a paso

Ejemplo: **«Mi metido en el hogar da una prenda (6)»**.

1. **Buscar la definición en una punta.** Al final dice «una prenda»: la respuesta es una prenda de 6 letras.
2. **Buscar el indicador.** «Metido en» sugiere que algo va dentro de otra cosa: es un contenedor.
3. **Identificar el material.** Lo que se mete es «Mi». Lo que lo contiene es «el hogar», que hay que traducir a un sinónimo: CASA.
4. **Hacer la operación.** Se mete MI dentro de CASA: CA + MI + SA = CAMISA.
5. **Comprobar.** CAMISA es una prenda y tiene 6 letras. Las dos lecturas coinciden.

Las palabras de unión como «da», «es» o «para» solo conectan las dos partes del enunuciado.

Tipos de juego de palabras

El sistema debe reconocer al menos estos tipos. Cada críptico del catálogo trae su tipo.

Tipo	Qué se hace	Indicadores típicos	Ejemplo
Anagrama	Reordenar las letras del material.	loco, revuelto, desordenado, alborotado, roto	Amor loco da una fruta del bosque (4): MORA
Oculta	La respuesta está escrita de corrido dentro del material.	esconde, oculta, guarda, en parte de	La cama tejida esconde la infusión argentina (4): MATE

Charada	Se pegan piezas una detrás de otra.	con, y, más, junto a (a veces ninguno)	Tour con el astro rey da una flor (7): GIRA + SOL
Contenedor	Una pieza va dentro de otra.	dentro de, metido en, abraza, rodea, traga	Mi metido en el hogar da una prenda (6): CA(MI)SA
Inversión	El material se lee al revés.	al revés, de vuelta, que vuelve, para atrás	Rama de vuelta para querer (4): AMAR
Supresión	Se quita una letra del principio, del final o del medio.	sin cabeza, descabezado, sin cola, sin corazón	Helado sin cabeza: corriente de agua (3): (F)RÍO
Homófono	Una palabra que suena igual a otra. En castellano: h muda, b/v, s/c/z, ll/y.	se oye, suena, de oído	Se oye como la cacería: hogar (4): CAZA suena CASA
Doble definición	Dos definiciones de la misma palabra. No hay indicador ni material.	(ninguno)	Felino o herramienta para cambiar la rueda (4): GATO
Iniciales y finales	Las primeras o últimas letras de palabras seguidas.	al principio, cabezas de, colas de, finales de	Al principio, ocho sapos observan un animal peludo (3): OSO
Sustitución	Se cambia una letra por otra.	con otra cabeza, con otro final	Cabello con otra cabeza cubre a la novia (4): PELO a VELO

Símbolos	Abreviaturas convencionales: números romanos, puntos cardinales, elementos químicos, notas musicales.	(según el símbolo)	Calcio y hierro para el desayuno porteño (4): Ca + Fe
Combinada	Dos o más operaciones en el mismo enunciado.	(los de cada operación)	Manco perturbado abraza a uno: senda (6): anagrama de MANCO con I adentro

Convenciones del castellano: «uno» puede valer I (romano); norte, sur, este y oeste valen N, S, E y O. La puntuación suele ser engañosa y se puede ignorar.

Para profundizar: guía de crípticos en castellano de mata.juegos y How to solve the cryptic crossword, de The Globe and Mail. Todos los ejemplos son del catálogo de la cátedra.

Sitio público (frontend)

El jugador no crea cuenta ni contraseña: en su primera visita elige un nombre y el sitio guarda un token en una cookie que lo identifica en las visitas siguientes. El nombre se puede cambiar en cualquier momento: el historial, la racha y los puntos se conservan, porque pertenecen al jugador y no al nombre. También puede elegir «empezar como otro jugador»: el sitio borra la cookie y crea un jugador nuevo, desde cero. Antes de hacerlo debe mostrar un aviso claro, porque el jugador anterior no se puede recuperar. Su historial queda en la base y sigue contando en la tabla de posiciones, pero ya nadie puede jugar con él.

Desde cualquier página tiene a mano cuatro secciones: **Jugar**, **Mi racha**, **Tabla de posiciones** y **Cómo se juega**, que abre el chatbot (o una página de instrucciones si no realizan el chatbot).

Reglas del nombre: entre 3 y 20 caracteres, único sin distinguir mayúsculas, sin espacios al principio ni al final. Si el nombre está tomado, se avisa y se sugiere una variante (por ejemplo, **`Pedrito_2`**). El admin puede bloquear nombres ofensivos.

Dónde se guarda cada dato

Todo lo que importa se guarda en la base de datos del servidor (nombres, puntos, partidas y rachas). El navegador solo guarda un token que dice quién es el jugador.

Lo que son preferencias visuales (por ejemplo, tema oscuro) pueden guardarlas si quieren en el **`localStorage`**.

No se guardan puntos en una cookie ni en **`localStorage`**. Si el jugador borra las cookies, pierde el acceso a su jugador y empieza de cero.

Partida

1. El jugador elige una dificultad: Fácil, Medio o Avanzado.
2. El sistema le asigna al azar un críptico activo de esa dificultad que todavía no haya jugado y ve un botón que dice **Jugar**.
3. Al presionar **Jugar**, recién ahí arranca el cronómetro y se le muestra el críptico y la enumeración. No las pistas.
4. El jugador escribe respuestas (tiene hasta 5 intentos posibles en total). Cada intento se valida en el servidor, y la respuesta correcta nunca viaja al navegador antes de resolver.
5. El jugador puede pedir pistas por críptico, pero esto le restará puntos (ver abajo).
6. Al acertar, el servidor registra, calcula el tiempo y los puntos, actualiza la racha y muestra la explicación.
7. El jugador puede rendirse: Hay un botón **Rendirse**, que revela la respuesta y la explicación y suma 0 puntos.

No hay tiempo límite. Si el jugador cierra la pestaña, la partida sigue abierta y el reloj sigue corriendo: al volver, retoma la misma partida. Cada críptico se juega una sola vez por jugador. El tiempo oficial es siempre calculado en el servidor.

La respuesta se compara normalizada: sin tildes, sin mayúsculas y sin espacios ni signos. Así, «jamon», «Jamón» y « JAMÓN » son correctas. La Ñ se distingue de la N.

Pistas escalonadas

El jugador puede pedir hasta cuatro pistas, siempre en este orden. Cada una resta puntos (ver Sistema de puntaje).

1. **Tipo:** el tipo de juego de palabras, por ejemplo «Anagrama».
2. **Definición:** se resalta la parte que es definición.
3. **Indicador y material:** se resaltan el indicador y el material.
4. **Ayuda:** el texto libre de ayuda del críptico.

Las pistas salen de los campos JSON de cada críptico.

Sistema de puntaje

Gana más quien resuelve más rápido, con menos pistas y sin cortar la racha. El tipo y la dificultad del críptico fijan los puntos base. El tiempo y las pistas los reducen, y la racha los aumenta.

```
puntos = base × tiempo × pistas × racha
```

El resultado se redondea al entero más cercano. Un críptico resuelto da como mínimo 10 puntos. Rendirse da 0 puntos.

1. Base

```
base = puntos del tipo × multiplicador de la dificultad
```

Los dos valores se configuran desde el backend (ver «Puntos de cada tipo de críptico»). Con los valores iniciales, la base es: 100 para fácil, 200 para medio y 300 para avanzado.

2. Factor de tiempo

Cada dificultad tiene un tiempo de referencia: 60 segundos para fácil, 180 para medio y 300 para avanzado.

- Si el jugador resuelve dentro del tiempo de referencia, el factor es 1 y no pierde nada.

- Si tarda más, el factor es tiempo de referencia ÷ tiempo que tardó. Por ejemplo, si tarda el doble, el factor es 0,5.

- El factor nunca baja de 0,25, aunque el jugador tarde horas.

El tiempo es el que pasa entre dos timestamps: *iniciado* y *resuelto*, y lo calcula el servidor.

3. Factor de pistas

Cada pista que pide el jugador resta 0.20

Pistas	0	1	2	3	4
Factor	1	0,8	0,6	0,4	0,2

4. Factor de racha

Cada día de racha suma 0.05, hasta un máximo de 1,5, que se alcanza a los 10 días. Se usa la racha contando el día de hoy, así que todos los crípticos que el jugador resuelve en un mismo día tienen el mismo bonus.

Días de racha	1	2	4	10 o más
Factor	1,05	1,10	1,20	1,50

Ejemplo

Un críptico medio, resuelto en 240 segundos con 1 pista, por un jugador que con hoy lleva 4 días de racha:

- Base: es medio, así que los puntos del tipo (100) se multiplican por 2.

100 × 2 = 200

- Tiempo: la referencia para medio es 180 segundos y tardó 240, o sea que se pasó. El factor es la referencia dividida por lo que tardó:

180 ÷ 240 = 0,75. Pierde un 25 % por tema tiempo.

- Pistas: usó 1 y cada pista resta 0,20.

1 − 0,20 = 0,8. Pierde un 20 % por la pista.

- Racha: lleva 4 días y cada día suma 0,05.

1 + 0,05 × 4 = 1,2. Gana un 20 % por la racha.

Total:

200 × 0,75 × 0,8 × 1,2 = **144 puntos**

Si lo hubiera resuelto en 2 minutos y sin pistas, habría ganado 200 × 1 × 1 × 1,2 = 240 puntos.

Puntos de cada tipo de críptico

Los puntos base no están fijos en el código. Se definen desde el backend en dos partes:

- Puntos por tipo: cada tipo de críptico tiene sus puntos. Por ejemplo, un anagrama puede valer más que una oculta. Eso lo definen ustedes como quieran.

- Multiplicador por dificultad: cada dificultad multiplica esos puntos. Por ejemplo, fácil ×1, medio ×2 y avanzado ×3.

Los valores iniciales los debería cargar db:seed: 100 puntos para todos los tipos y multiplicadores 1, 2 y 3 por defecto.

Reglas:

- Los puntos de un tipo son un número entero entre 10 y 1000.

- El multiplicador de dificultad es un número mayor que 0 y menor o igual a 5.

- Un cambio solo afecta a las partidas que se resuelvan después. Cada partida guarda sus puntos al resolverse, así que las partidas ya resueltas no se recalculan y la tabla de posiciones no cambia.

Backend de administración

Al backend entran usuarios con email y contraseña. Cada uno tiene uno de estos dos roles:

- **Administrador**: hace todo lo que describe esta sección. Además, da de alta, edita y desactiva a los otros usuarios del backend y les asigna el rol.

  - **Crípticos:** listar con filtros por dificultad, tipo, estado y origen, ver estadísticas de cada uno si existen (veces jugado, porcentaje resuelto, tiempo promedio), activar o desactivar.
  - **Configuración:** Editar los puntos por tipo y los multiplicadores, la URL de la fuente de crípticos y el texto de bienvenida.
  - **Sincronización:** un botón «Sincronizar ahora» que trae el catálogo de la fuente, los guarda en la BBDD y muestra un informe de cómo resultó la sincronización.
  - **Jugadores:** listar con su puntaje y su racha, bloquear nombres ofensivos y reiniciar la tabla de posiciones.

- **Editor de puntajes**: solo puede ver los puntos de cada tipo y los multiplicadores y editarlos. No puede sincronizar, editar cripticos, ver jugadores ni administrar usuarios.

El primer admin se crea con una tarea rake o desde `db/seeds.rb`. No hay registro público de admins.

Un admin puede crear otros admins.

Los permisos se controlan en el servidor, en cada controlador. Ocultar los links del menú no alcanza: si un editor de puntajes escribe a mano la URL de otra página del backend, el sistema lo rechaza.

Para el login se puede usar el generador de autenticación que trae Rails 8 (`bin/rails generate authentication`).

La API de cripticos

La cátedra publica el catálogo como una API JSON de solo lectura. El repositorio está en github.com/ecorrons/cryptics-api, y su README explica cómo usarla.

Ruta	Contenido
`/v1/cryptics.json`	El catálogo completo.
`/v1/version.json`	Solo la versión vigente del catálogo.

`/v1/cryptics_with_errors.json`	Un catálogo de prueba con errores a propósito.

Hay tres formas de consumirla. Las rutas son las mismas: solo cambia la URL base, que se configura desde el backend.

1. **La API de la cátedra**, publicada en GitHub Pages: **`https://ecorrons.github.io/cryptics-api/v1/cryptics.json`.**
2. **Hacer una copia propia en GitHub:** un fork del repositorio con GitHub Pages activado, y entonces pueden agregar sus crípticos propios.
3. **Un servidor local** levantado con WEBrick desde una copia del repositorio, para desarrollar sin conexión, y también agregar sus crípticos al catálogo.

En todos los casos, la sincronización se hace por HTTP. No vale leer el archivo directamente del disco. El catálogo con errores sirve para probar que la sincronización los detecta e informa.

Los cripticos se guardan en la base de datos del sistema y el juego siempre los lee de ahí. La API solo se consulta al sincronizar, cuando un admin lo pide. La carga inicial (db:seed) usa el mismo sincronizador.

Reglas de la sincronización

1. Se hace un **`GET`** a la URL configurada, con timeout de 10 s. Si falla la red o el JSON no parsea, no se toca nada y se informa el error.
2. Si el campo **`version`** coincide con el de la última sincronización exitosa, se informa «Sin cambios» y termina.
3. Cada críptico se identifica por su **`id`** de la fuente. Si no existe, se crea; si existe y cambió, se actualiza.
4. Un críptico inválido se saltea y se informa con su **`id`** y el motivo. Los demás se procesan igual.
5. Un críptico que ya no viene en la fuente no se borra, porque puede tener partidas jugadas: se desactiva.
6. Cada sincronización queda registrada con fecha, admin que la hizo, versión y cantidad de creados, actualizados, desactivados y rechazados.

La lógica va en un objeto de servicio (por ejemplo, **`SincronizadorDeCripticos`**), NO dentro del controlador. No metan lógica de más en los controladores.

Validaciones de un críptico

- **`enunciado`, `respuesta`, `tipo` y `dificultad`** son obligatorios. **`tipo`** y **`dificultad`** pertenecen a los valores definidos.
- La enumeración coincide con el largo de la respuesta, sin contar tildes ni espacios.
- Cada definición, indicador y material aparece textualmente en el enunciado.

- La doble definición lleva 2 definiciones
- Si el tipo es **oculta**, la respuesta está dentro del material, sin espacios. Si es **anagrama**, el material tiene exactamente las mismas letras que la respuesta. Si es **iniciales** o **finales**, las primeras o últimas letras del material forman la respuesta.

Las dos últimas reglas hacen que el sistema verifique cripticos por sí mismo, y son la parte más interesante de modelar.

Tabla de posiciones

La tabla de posiciones es pública: la ven todos los jugadores desde el menú, incluso antes de elegir un nombre. Ordena por puntos y, en caso de empate, por menor tiempo promedio.

- **Columnas:** posición, nombre, puntos, # cripticos resueltos, tiempo promedio, porcentaje resuelto sin pistas, racha actual e insignia de racha más alta.
- **Pestañas:** «Esta semana», «Histórico» y «Mejores rachas», que ordena por racha mejor racha (activa).
- **Filtro** por dificultad en las pestañas de puntos.
- **El jugador actual** aparece resaltado y, si no está en el top 20, se muestra su posición al pie.
- Los jugadores bloqueados no aparecen.

Extras obligatorios para grupos de 4 y 5 integrantes

- **Críptico del día:** un críptico igual para todos cada día, elegido de forma determinística a partir de la fecha, que vale el doble.
- **Compartir resultado:** un texto para WhatsApp que no revela la respuesta.

Extra obligatorio para grupos de 5 integrantes

- **Módulo de estadísticas de juego**: visible en el backend, para los administradores. Consultas del estilo tipo de juego más jugado en los últimos X días, porcentaje de cripticos resueltos por usuario, y otras estadísticas que se les ocurran. Pueden mostrar las querys de forma gràfica.

Bonus track: Chatbot de reglas

El sitio incluye un asistente en forma de chat que enseña a jugar paso a paso. Es un **chatbot guiado**: una conversación con nodos y botones de respuesta, sin inteligencia

artificial (sin LLM) y es opcional (para tener mas nota, si no lo quieren hacer, pueden poner simplemente una pagina con instrucciones de juego).

- **Estructura:** cada nodo tiene un mensaje del bot y opciones que llevan a otros nodos. Los nodos se definen en un archivo YAML o en la base, editables desde el admin.
- **Recorrido mínimo:** bienvenida, qué es un críptico, las partes de un enunciado, un nodo por cada tipo de juego de palabras, cómo funcionan el puntaje, las pistas, y las rachas, y vuelta al menú desde cualquier punto.
- **Interfaz:** un panel que se abre desde cualquier página.
- **Estado:** el nodo actual vive en la sesión de Rails o en el cliente, nunca en la cookie del token del jugador.
- **Contenido:** los ejemplos del chatbot no pueden ser crípticos activos del catálogo, para no regalar respuestas.

Anexo: formato del JSON de cripticos

La API de la cátedra (github.com/ecorrons/cryptics-api) usa este formato en **`v1/cryptics.json`**. La estructura NO va a cambiar. El catálogo trae 34 crípticos: 12 Fácil, 13 Medio y 9 Avanzado.


```
{
  "version": "2026-10-19.1",
  "generado": "2026-10-19",
  "fuente": "Cátedra de Ruby on Rails",
  "cripticos": [
    {
      "id": "cr-013",
      "enunciado": "Mi metido en el hogar da una prenda",
      "enumeracion": "(6)",
      "respuesta": "CAMISA",
      "definiciones": ["una prenda"],
      "indicadores": ["metido en"],
      "material": ["Mi", "el hogar"],
      "tipo": "contenedor",
      "dificultad": "medio",
      "ayuda": "El hogar es una palabra de cuatro letras. Partila al medio y meté MI.",
      "explicacion": "MI dentro de CASA (el hogar): CA + MI + SA = CAMISA.",
      "autor": "Cátedra"
    }
  ]
}
```


Campo	Tipo	Descripción

`version`	texto	Cambia en cada publicación. Si no cambió, no hay nada que sincronizar.
`id`	texto	Identificador estable del críptico en la fuente. Nunca se reutiliza.
`enunciado`	texto	La frase que ve el jugador, sin la enumeración.
`enumeracion`	texto	Cantidad de letras entre paréntesis.
`respuesta`	texto	En mayúsculas, con tildes.
`definiciones`	lista de textos	Fragmentos del enunciado. Dos en una doble definición.
`indicadores`	lista de textos	Fragmentos del enunciado. Vacía en una doble definición.
`material`	lista de textos	Fragmentos del enunciado sobre los que se opera.
`tipo`	texto	anagrama, oculta, charada, contenedor, inversion, supresion, homofono, doble_definicion, iniciales, finales, sustitucion, simbolos o combinada.
`dificultad`	texto	facil, medio o avanzado.
`ayuda`	texto	La cuarta pista.

`explicación`	texto	Se muestra al resolver o rendirse.
`autor`	texto	Quién escribió el críptico.

Las listas son siempre listas, aunque tengan un solo elemento o ninguno, para que el código no tenga que distinguir casos.

Más sugerencias técnicas

- Ruby 3.3 o superior y Rails 8. La base puede ser SQLite, la que trae Rails 8 por defecto.
- Se recomienda usar Hotwire (Turbo y Stimulus) para la interactividad. No está permitido usar un framework de frontend aparte (tipo Vue o React).
- Deseable pero no obligatorio si quieren hacer Tests de unidad y funcionales con Minitest o RSpec: modelos, servicios (puntaje, sincronización).
- **bin/rails db:setup** deja el sistema listo para jugar, con el JSON semilla cargado y un admin creado.
- Un **`README`** con instrucciones para levantar el proyecto, las credenciales del admin de prueba y las decisiones de diseño (gruesas) que tomaron para hacer el TFI.
- Repositorio Git con commits significativos **de todos los integrantes**. Si alguien no tiene commits no podrá ser evaluado.
- Interfaz en español, usable en celular/tablets (responsive).

Criterios de evaluación

Se evaluará que el juego, el puntaje, las rachas, el ranking, el backend cumplan como mínimo con lo pedido en este enunciado.

Asociaciones, validaciones (incluidas las de tipo) a nivel modelo y a nivel BBDD, lógica fuera de los controladores (no violar el modelo MVC).

Manejo de errores de red y datos inválidos, idempotencia, informe. Manejo de excepciones

Tiempo y respuesta calculados en el servidor, cookie firmada, admin protegido, restricciones en la base.

Convenciones de Rails (convention over configuration), commits claros y repartidos entre todos los miembros del grupo, código prolijo.

Buen compañerismo y trabajo en equipo (soft skills).

La defensa oral puede subir o bajar la nota individual. Si bien la entrega es grupal, si alguien no se presenta al coloquio, no podra aprobar la cursada.

Consideraciones generales

- Se espera que todos los modelos tengan las validaciones adecuadas para asegurar la integridad de los datos.
- Se deberá incluir un conjunto de datos de prueba para poder evaluar el sistema. Estos datos deberán generarse mediante el script db/seeds.rb del proyecto RoR.
- Se espera que la interfaz de usuario sea cómoda, entendible, y lo más intuitiva posible, pero no se solicitará para la entrega ningún tipo de librería ni interfaz en particular. Debería ser un diseño responsive, que se visualice bien en distintos dispositivos (desktop y mobile).
- Se deberá documentar en el archivo README.md del raiz del proyecto cualquier decisión de diseño importante que se haya tomado, así como también se deberán dejar escritos los requisitos técnicos y los pasos (comandos) para hacer funcionar la aplicación de manera local en sistemas operativos tipo *nix y Windows.
- La funcionalidad solicitada en este enunciado es la mínima que debe implementarse de base para poder aprobar el trabajo. Podrán agregar más funcionalidad a la aplicación (el Chatbot, y ver mencionados en el apartado Extras), siempre y cuando respeten lo solicitado en este documento. No se permitirá el intercambio de funcionalidades solicitadas por otras que no lo estén.

Modalidad de la entrega

El TP es grupal y tiene una sola entrega, aproximadamente seis semanas después de publicado este enunciado. La cátedra confirmará la fecha exacta, que sera en diciembre. Se entrega el link al repositorio del grupo por medio de una tarea de Moodle.

Después de la entrega hay una defensa (coloquio) oral del grupo. Cada integrante explica una parte del código elegida por los docentes, **así que todos tienen que conocer el sistema completo**. El historial de Git debe mostrar aportes de todos los integrantes.

Los trabajos entregados se compararán entre grupos para encontrar posibles similitudes en su estructura que pudieran provenir de copias entre distintas entregas. Esta entrega es de caracter obligatorio, y su aprobación (en cualquiera de sus instancias) es condición necesaria para aprobar la cursada de la materia. El trabajo tiene que estar versionado en un repositorio Git de GitHub **de todos** los participantes del grupo, cuya URL deberán entregarnos mediante una tarea dedicada que publicaremos en el curso de la plataforma de

Cátedras (Moodle). Quienes sean parte del grupo deben contar con commits y participación en el repositorio.

En caso que el repositorio sea privado, vamos a necesitar que nos brinden acceso al mismo para poder descargarlo y evaluarlo. Si esta es su decisión, por favor consultá antes de la fecha límite de la entrega con los docentes para saber a qué usuarios de GitHub deberán brindarles acceso a sus repositorios. Las fechas límite de entrega del trabajo fueron publicadas en el curso de la plataforma de Cátedras. No se admitirán entregas fuera de término.

Aunque NO lleguen a la fecha de Diciembre, deben entregar un link a un repositorio Git aunque sea en blanco, pero entreguenlo y serán citados al coloquio oral. Si NO lo entregan, PIERDEN la oportunidad de Reentrega en febrero y por ende, pierden la cursada.